



Buy Now > X



EN

BLOGS ▾

category ▾

[Home](#) > [Blog](#) > [Data Engineering](#)

Avro vs. Parquet: A Complete Comparison for Big Data Storage

A detailed comparison of Avro and Parquet, covering their architecture, use cases, performance, and how they fit into modern big data workflows.

[☰ Contents](#)

Feb 26, 2025 · 15 min read



Tim Lu

Data scientist with a passion for sharing industry knowledge.

TOPICS

[Data Engineering](#)[Big Data](#)

Efficient data storage is a critical part of any big data system. Avro and Parquet are two widely used formats, each designed for different workloads—Avro excels in streaming and schema evolution, while Parquet is optimized for analytics and storage efficiency. Understanding their differences is essential for building scalable data pipelines.

This guide breaks down their architecture, use cases, and how they fit into [modern data workflows](#).

What is Avro?

Avro is a row-based storage format developed for the Apache Hadoop project. It is designed to efficiently serialize data for exchange between systems, making it particularly useful for **streaming data platforms** and distributed applications.

Avro defines schemas using JSON for human readability, but the actual data is stored in a compact binary format for efficiency. This design enables both easy schema management and fast serialization.

Features of Avro

Avro offers several advantages, particularly in terms of schema evolution and storage efficiency:

- **Schema evolution:** Avro embeds schema metadata within the data, enabling seamless schema evolution. This means new fields can be added or existing fields modified without requiring a full rewrite of the dataset, making Avro highly flexible for data pipelines.
- **Compact serialization:** Avro's binary serialization minimizes storage overhead and enhances performance in data exchange scenarios. It is especially useful in environments where efficient serialization and deserialization are crucial, such as messaging queues and data streams.

Use cases for Avro

- **Streaming and messaging systems** – Commonly used in **Apache Kafka** for efficient event serialization.
- **Data exchange and interoperability** – Ideal for sharing structured data across different applications.
- **Row-oriented storage** – Works well for workloads where data needs to be written and read sequentially in rows.

What is Parquet?

Parquet is a columnar storage format optimized for high-performance analytical workloads. Unlike row-based formats like Avro, Parquet stores data column-wise, making it significantly more efficient for big data analytics. Developed by Apache, Parquet is widely used in data warehouses and distributed computing frameworks such as **Apache Spark** and Hadoop.

Features of Parquet

Parquet is designed for analytical performance, offering key benefits such as:

- **Columnar storage:** Since Parquet stores data column-wise, queries can efficiently scan only the required columns instead of loading entire rows. This reduces disk I/O, leading to faster query performance, especially in read-heavy workloads.
- **Efficient compression:** Parquet employs advanced compression techniques such as dictionary encoding, run-length encoding, and bit-packing, reducing storage costs while maintaining high query speeds. Because similar data types are stored together, Parquet achieves better compression than row-based formats.

Use cases for Parquet

- **Data warehousing and analytics** – Used in platforms like Amazon [Redshift](#), Google [BigQuery](#), and [Snowflake](#).
- **Big data processing** – Optimized for distributed computing frameworks such as Apache Spark and Presto.
- **Efficient query performance** – Ideal for read-heavy workloads where only specific columns are needed.

To dive deeper into Parquet and learn how to work with it in practice, check out this [Apache Parquet tutorial](#).

Become a Data Engineer

Become a data engineer through advanced Python learning

Start Learning for Free

Differences Between Avro and Parquet

Avro and Parquet are widely used data storage formats in big data ecosystems, but they serve different purposes and excel in different scenarios. Below is a detailed comparison of their differences.

Data structure

- **Avro:** Uses a row-based storage format, meaning entire records are stored sequentially. This makes Avro efficient for write-heavy workloads where data needs to be quickly appended.
- **Parquet:** Uses a columnar storage format, where data is stored by columns rather than rows. This structure is beneficial for analytical queries that require reading only specific columns rather than entire rows.

Visualizing row-based vs. columnar storage

To better understand the difference between row-based (Avro) and columnar-based (Parquet) storage, consider this dataset:

ID	Name	Age	City
101	Alice	25	New York
102	Bob	30	Chicago
103	Carol	28	Seattle

How Avro stores data (row-based):

- [101, Alice, 25, New York]
- [102, Bob, 30, Chicago]
- [103, Carol, 28, Seattle]

Each record is stored sequentially, making it efficient for write operations but slower for querying specific fields.

How Parquet stores data (columnar-based):

- ID: [101, 102, 103]
- Name: [Alice, Bob, Carol]
- Age: [25, 30, 28]

- **City:** [New York, Chicago, Seattle]

Each column is stored separately, making it faster to retrieve only the required columns (e.g., querying just the "Age" column).

Schema evolution

- **Avro:** Designed for schema evolution, allowing new fields to be added or modified without breaking compatibility with older data. The schema is stored with the data, making it self-descriptive.
- **Parquet:** Supports schema evolution but is less flexible than Avro. Schema changes can be more complex, especially when adding or modifying column structures.

Compression and storage efficiency

- **Avro:** Uses compact binary encoding, but it does not leverage columnar compression techniques, leading to larger file sizes when compared to Parquet.
- **Parquet:** Uses columnar compression techniques like dictionary encoding, run-length encoding, and bit-packing, making it more space-efficient, especially for large datasets.

Query performance

- **Avro:** Not optimized for analytical queries since it stores data row-wise. Scanning large datasets requires reading entire records, leading to slower query performance in analytics workloads.
- **Parquet:** Optimized for fast queries, especially when only a subset of columns is needed. Its columnar structure allows for selective scanning, improving performance in big data analytics.

Write and read efficiency

- **Avro:** Provides fast write speeds since it stores data row-by-row. However, read operations can be slower for analytics because entire rows must be read.
- **Parquet:** Optimized for read performance but can have slower write speeds due to the overhead of columnar storage and compression techniques.

Avro vs. Parquet Comparison Table

Feature	Avro	Parquet
Storage Format	Row-based (stores entire records sequentially)	Columnar-based (stores data by columns)
Best For	Streaming, event data, schema evolution	Analytical queries, big data analytics
Schema Evolution	Excellent – schema is stored with data, allowing seamless updates	Limited – schema evolution is possible but requires careful handling
Compression	Compact binary encoding but less optimized for analytics	Highly compressed using columnar compression techniques (dictionary encoding, run-length encoding, bit-packing)
Read Performance	Slower for analytics since entire rows must be read	Faster for analytics as only required columns are read
Write Performance	Faster – appends entire rows quickly	Slower – columnar storage requires additional processing
Query Efficiency	Inefficient for analytical queries due to row-based structure	Highly efficient for analytical queries since only required columns are scanned
File Size	Generally larger due to row-based storage	Smaller file sizes due to better compression techniques

Use Cases	Event-driven architectures, Kafka messaging systems, log storage	Data lakes, data warehouses, ETL processes, analytical workloads
Processing Frameworks	Works well with Apache Kafka, Hadoop, Spark	Optimized for Apache Spark, Hive, Presto, Snowflake
Support for Nested Data	Supports nested data, but requires schema definition	Optimized for nested structures, making it better suited for hierarchical data
Interoperability	Widely used in streaming platforms	Preferred for big data processing and analytical workloads
File Extension	.avro	.parquet
Primary Industry Adoption	Streaming platforms, logging, real-time pipelines	Data warehousing, analytics, business intelligence

When to Use Avro vs Parquet

Choosing between Avro and Parquet depends on your specific use case, workload characteristics, and data processing requirements. Below are practical guidelines to help you determine when to use each format.

When to use Avro

Avro is best suited for scenarios that require efficient serialization, schema evolution, and row-based storage. Consider using Avro in the following situations:

- **Streaming and event-driven data pipelines:** Avro is widely used in real-time data streaming platforms like Apache Kafka due to its compact binary format and

efficient serialization.

- **Schema evolution is frequent:** Avro is the preferred choice when dealing with evolving data schemas since it embeds the schema within the file, allowing smooth modifications without breaking compatibility.
- **Inter-system data exchange and integration:** Avro is commonly used to transmit structured data between different applications due to its self-descriptive schema and support for multiple programming languages.
- **Write-heavy workloads:** If your workflow involves frequent writes or appends, Avro performs better than Parquet. It stores data in sequential rows without the overhead of columnar indexing.
- **Log storage and raw data collection:** Avro is often used in logging systems or storing raw unprocessed data because of its efficient row-wise storage and ability to capture detailed records.

When to use Parquet

Parquet is ideal for analytical workloads, big data processing, and storage efficiency. Use Parquet when:

- **Analytical queries and data warehousing:** If your primary goal is fast query performance in data lakes, data warehouses, or **OLAP** (Online Analytical Processing) systems, Parquet's columnar structure is highly efficient.
- **Read-heavy workloads:** Parquet is optimized for scenarios where data is read frequently but written less often, such as BI dashboards, reporting, and batch analytics.
- **Big data and distributed processing:** Parquet is the preferred format for big data frameworks like Apache Spark, Hive, Presto, and Snowflake, where columnar storage reduces I/O costs and improves performance.
- **Compression and storage optimization:** If storage efficiency is a priority, Parquet's advanced compression techniques (e.g., dictionary encoding, run-length encoding) significantly reduce file sizes compared to row-based formats.
- **Selective column reads and schema projection:** When working with large datasets but only querying a few columns, Parquet enables selective column scanning, making queries significantly faster and more cost-efficient.

Which one should you choose?

Scenario	Recommended Format
Real-time streaming (Kafka, logs, messaging)	✓ Avro
Frequent schema changes / evolving data structure	✓ Avro
Row-based storage for fast writes	✓ Avro
Data exchange between applications	✓ Avro
Big data analytics and querying	✓ Parquet
Efficient storage and compression	✓ Parquet
Read-heavy workloads (data lakes, warehousing)	✓ Parquet
Columnar-based querying and filtering	✓ Parquet

How Avro and Parquet Work with Big Data Tools

Avro and Parquet are widely used in **big data processing frameworks**, cloud platforms, and distributed computing environments. Below is an overview of how Avro and Parquet integrate with popular big data tools.

Apache Spark

Apache Spark is a distributed computing framework widely used for processing large datasets. It supports both Avro and Parquet, but each format has distinct advantages depending on the use case.

Using Avro in Apache Spark

- Avro is ideal for data ingestion and ETL pipelines due to its row-based structure.
- It allows schema evolution, making it a preferred choice for Kafka streaming pipelines.
- Typically used as an intermediary format before converting data into analytics-friendly formats like Parquet.

Example:

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("AvroExample").getOrCreate()

# Read Avro file
df = spark.read.format("avro").load("data.avro")
df.show()

# Write DataFrame to Avro
df.write.format("avro").save("output.avro")
```



POWERED BY  datalab

Using Parquet in Apache Spark

- Parquet is optimized for analytics and batch processing in Spark.
- Since Spark queries often involve scanning specific columns, Parquet's columnar storage reduces I/O and speeds up queries.
- It is the default storage format in many data lakes and analytical environments.

Example:

```
# Read Parquet file
df = spark.read.parquet("data.parquet")
df.show()

# Write DataFrame to Parquet
df.write.mode("overwrite").parquet("output.parquet")
```



POWERED BY  datalab

Apache Hive and Presto

Apache Hive and Presto are SQL-based query engines designed for big data. Both engines support Avro and Parquet but behave differently:

Avro in Hive and Presto

- Used in data ingestion workflows before transformation into an optimized format.
- Provides flexibility for schema evolution, but queries can be slower due to row-based scanning.

Example:

```
CREATE EXTERNAL TABLE avro_table  
STORED AS AVRO  
LOCATION 's3://my-data-bucket/avro/';
```



POWERED BY  datalab

Parquet in Hive and Presto

- Preferred format for analytics due to column pruning and efficient compression.
- Queries are significantly faster because only necessary columns are read.

Example:

```
CREATE EXTERNAL TABLE parquet_table  
STORED AS PARQUET  
LOCATION 's3://my-data-bucket/parquet/';
```



POWERED BY  datalab

Apache Kafka

Kafka is a real-time data streaming platform, and Avro is the de facto standard for message serialization.

Why Avro is preferred in Kafka

- Compact binary format makes messages smaller, reducing bandwidth and storage

costs.

- Schema evolution support ensures compatibility between producers and consumers.
- Works seamlessly with Confluent Schema Registry, allowing schema version control.

Example:

```
from confluent_kafka import avro
from confluent_kafka.avro import AvroProducer

schema_registry_url = "http://localhost:8081"
avro_producer = AvroProducer({'bootstrap.servers': 'localhost:9092',
                              'schema.registry.url': schema_registry_url},
                              default_value_schema=avro.loads(value_schema_str))

avro_producer.produce(topic='avro_topic', value={"id": 1, "name": "Alice"})
avro_producer.flush()
```



POWERED BY  datalab

Cloud data platforms (AWS, GCP, Azure)

Both Avro and Parquet are supported by major cloud platforms but are used differently.

AWS (Amazon S3, Glue, Redshift, Athena)

- **Avro:** Used for storing streaming data and schema evolution in [AWS Glue](#) ETL jobs.
- **Parquet:** Preferred in [AWS Athena](#), Redshift Spectrum, and data lakes for faster analytics.

```
SELECT * FROM my_parquet_table WHERE year = 2023;
```



POWERED BY  datalab

Google Cloud Platform (BigQuery, Dataflow, GCS)

- **Avro:** Used to ingest raw data in Google Dataflow.
- **Parquet:** Optimized for Google BigQuery, allowing column-based retrieval.

```
LOAD DATA INTO my_dataset.my_table
FROM 'gs://my-bucket/data.parquet'
FORMAT PARQUET;
```

POWERED BY  datalab

Azure (Azure Data Lake, Synapse Analytics, Databricks)

- **Avro:** Used for data exchange and ingestion in [Azure Data Factory](#).
- **Parquet:** Preferred in [Azure Synapse Analytics](#) and Azure Databricks for optimized storage and analytics.

ETL pipelines and data warehousing

Both formats play different roles in ETL pipelines:

ETL Stage	Best Format	Reason
Ingestion (Streaming and Logs)	✓ Avro	Efficient for real-time data ingestion (Kafka, IoT, event logs).
Intermediate Processing	✓ Avro	Schema evolution allows data transformation without breaking pipelines.
Final Storage (Analytics and BI)	✓ Parquet	Faster queries and optimized storage for columnar retrieval.

Conclusion

Avro and Parquet are essential data storage formats in big data ecosystems, each serving different purposes. This post explored their technical architecture, use cases, integration with big data tools, and role in ETL pipelines.

Key takeaways include:

- Avro's row-based format is efficient for streaming, schema evolution, and data serialization.
- Parquet's columnar format optimizes analytical queries, compression, and storage efficiency.
- Big data tools such as Apache Spark, Hive, and Kafka integrate with these formats in different ways.
- ETL pipelines often use both, with Avro handling raw data ingestion and Parquet enabling efficient analytics.

To build a stronger foundation in data warehousing and big data processing, explore [Data Warehousing Concepts](#). For insights into real-time vs. batch data processing, read [Batch vs. Stream Processing](#). To get hands-on experience with big data tools, the [Big Data Fundamentals with PySpark](#) course is a great place to start.

Understanding these storage formats and their applications is key to designing scalable and efficient data architectures.

Become a Data Engineer

Prove your skills as a job-ready data engineer.

Fast-Track My Data Career

FAQs

Why is Parquet preferred for analytics?



Parquet's columnar format makes it highly efficient for queries that access only specific columns. It reduces overhead and provides excellent compression for repetitive data, making it ideal for data warehouses and business intelligence tools.

What are the storage differences between Avro and Parquet?



Which format offers better compression?



Can Avro and Parquet be used together?



How do Avro and Parquet integrate with big data tools?



AUTHOR

Tim Lu

in

I am a data scientist with experience in spatial analysis, machine learning, and data pipelines. I have worked with GCP, Hadoop, Hive, Snowflake, Airflow, and other data science/engineering processes.

TOPICS

Data Engineering Big Data



Training more people?

Get your team access to the full DataCamp for business platform.

For Business

For a bespoke solution [book a demo](#).

Learn more about data engineering with these

courses!

🚩 TRACK

Data Engineer in Python

🕒 40 hr

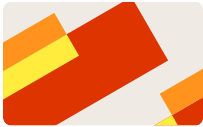
Gain in-demand skills to efficiently ingest, clean, manage data, and schedule and monitor pipelines, setting you apart in the data engineering field.

See Details →

Start Course

See More →

Related



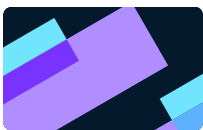
BLOG

Kafka vs RabbitMQ: Key Differences & When to Use Each



BLOG

BigQuery vs Redshift: Comparing Costs, Performanc...



BLOG

Apache Iceberg vs Delta Lake: Features, Differences & Use...

See More →

Grow your data skills with DataCamp for Mobile

Make progress on the go with our mobile courses and daily 5-minute coding challenges.



LEARN

Learn Python

Learn AI

Learn Power BI

Learn Data Engineering

Assessments

Career Tracks

Skill Tracks

Courses

Data Science Roadmap

DATA COURSES

Python Courses

R Courses

SQL Courses

Power BI Courses

Tableau Courses

Alteryx Courses

Azure Courses

AWS Courses

Google Sheets Courses

Excel Courses

AI Courses

Data Analysis Courses

Data Visualization Courses

Machine Learning Courses

Data Engineering Courses

Probability & Statistics Courses

DATALAB

Get Started

Pricing

Security

Documentation

CERTIFICATION

Certifications

Data Scientist

Data Analyst

Data Engineer

SQL Associate

Power BI Data Analyst

Tableau Certified Data Analyst

Azure Fundamentals

AI Fundamentals

RESOURCES

Resource Center

Upcoming Events

Blog

Code-Alongs

Tutorials

Docs

Open Source

RDocumentation

Book a Demo with DataCamp for Business

Data Portfolio

PLANS

Pricing

For Students

For Business

For Universities

Discounts, Promos & Sales

[Expense DataCamp](#)

[DataCamp Donates](#)

FOR BUSINESS

[Business Pricing](#)

[Teams Plan](#)

[Data & AI Unlimited Plan](#)

[Customer Stories](#)

[Partner Program](#)

ABOUT

[About Us](#)

[Learner Stories](#)

[Careers](#)

[Become an Instructor](#)

[Press](#)

[Leadership](#)

[Contact Us](#)

[DataCamp Español](#)

[DataCamp Português](#)

[DataCamp Deutsch](#)

[DataCamp Français](#)

SUPPORT

[Help Center](#)

[Become an Affiliate](#)



[Privacy Policy](#)

[Cookie Notice](#)

[Do Not Sell My Personal Information](#)

[Accessibility](#)

[Security](#)

[Terms of Use](#)

© 2026 DataCamp, Inc. All Rights Reserved.